IoT-Based Obstacle Detection System Using an Ultrasonic Sensor and ThingSpeak Cloud

Internet of Things (Cloud/Edge) | Experiential Learning Component (ELC)
Department of Computer Science and Engineering, Thapar Institute of Engineering and Technology, Patiala

1. Objective

The objective of this activity was to create a visualization of obstacle detection using an ultrasonic sensor for a robot or vehicle prototype. The task was to measure the distance in front of a moving platform, stream the data to a dashboard, show when the platform is approaching an obstacle, and use graphics or icons to represent detected obstacles so that observers can see how the system reacts in real time.

To meet this, we built an ESP32-based system that measures distance with an HC-SR04 ultrasonic sensor, classifies it as safe, warning or danger on the device itself, sounds a buzzer in the danger zone, uploads the readings to the ThingSpeak cloud for history graphs, and serves a live dashboard with a robot icon, obstacle icons and colour-coded alerts.

2. Components Used

Component	Purpose
ESP32 Dev Module	Microcontroller with built-in WiFi; reads the sensor, runs the logic and hosts the dashboard
HC-SR04 ultrasonic sensor	Measures the distance to obstacles in front of the robot
Active buzzer (TMB12A12)	Audible alert when an obstacle is too close
3 resistors (7.6 kΩ each)	Voltage divider on the Echo pin (5 V to about 3.3 V)
Breadboard and jumper wires	Building the circuit without soldering
Phone hotspot (2.4 GHz)	WiFi network for the ESP32, the laptop and internet access
Arduino IDE 2.3.10	Writing, compiling and uploading the code
ThingSpeak (MathWorks)	Cloud storage and graphs of the sensor data
Web browser (Chrome)	Viewing the live dashboard

3. System Architecture

```
HC-SR04 sensor --> ESP32 --> WiFi (hotspot) --+--> ThingSpeak cloud (history graphs)
                     |                        |
                     |                        +--> Live web dashboard (hosted by the ESP32)
                     |
                     +--> Buzzer (instant local alert)
```

The design splits the work between the edge and the cloud:

- **Edge (on the ESP32):** the distance is measured about 10 times per second and the status is decided on the device. The buzzer reacts immediately even if the internet is slow or down. The ESP32 also hosts the live dashboard on the local network.
- **Cloud (ThingSpeak):** distance and status are uploaded every 16 seconds, stored, and plotted as history graphs that can be viewed from anywhere.

The decision logic is a simple threshold rule, not a machine-learning model.

4. Working Principle

The HC-SR04 sends a short ultrasonic pulse when the TRIG pin is triggered, and the ECHO pin stays high for as long as the sound takes to travel to the obstacle and back. The ESP32 measures this time with `pulseIn()` and calculates:

Distance (cm) = (echo time in µs × 0.034) / 2.

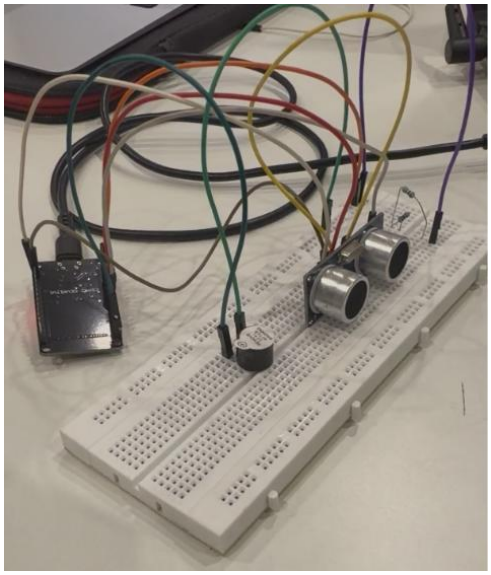
The speed of sound is about 0.034 cm per µs, and the result is divided by 2 because the pulse travels to the obstacle and back. Readings are limited to 200 cm, and no echo is reported as 200.

Distance	Status	Dashboard colour	Action
60 cm or more	0 - Safe	Green, "Path clear"	None
30 to 60 cm	1 - Warning	Yellow, "Approaching obstacle"	Yellow alert
Under 30 cm	2 - Danger	Red, "STOP! Obstacle very close"	Buzzer sounds, obstacle icon appears

5. Circuit Connections

Sensor / part	ESP32 pin
HC-SR04 VCC	VIN (5 V)
HC-SR04 GND	GND
HC-SR04 TRIG	GPIO 5
HC-SR04 ECHO	Resistor, then GPIO 18 (divider to GND)
Buzzer (+)	GPIO 23
Buzzer (-)	GND

The Echo pin outputs 5 V, but ESP32 GPIO pins tolerate only 3.3 V. One resistor sits between Echo and GPIO 18, and two resistors in series run from GPIO 18 to GND. This divides 5 V to about 3.3 V and protects the ESP32.



6. Implementation Steps

1. **Wiring and sensor test:** connected to the sensor and uploaded a basic sketch that prints the distance to the Serial Monitor. The values changed correctly as a hand moved in front of the sensor.
2. **WiFi connection:** configured the ESP32 to join the phone hotspot (2.4 GHz). The Serial Monitor prints the IP address once connected.
3. **ThingSpeak channel:** created a channel named "Robot Obstacle Detection" with Field 1 = Distance and Field 2 = Status and used its Write API Key in the code.
4. **Cloud upload:** the ESP32 sends the readings to ThingSpeak over HTTP every 16 seconds (the free tier allows one update per 15 seconds).
5. **Live dashboard:** the ESP32 runs a web server. A browser on the same network opens the ESP32's IP address and sees the radar view, status banner, distance, closest distance and danger-event count, refreshed every 300 ms.
6. **Buzzer:** Added the buzzer so the danger zone can be heard as well as seen.

7. Code Explanation

The full program is a single Arduino sketch. Its main parts are:

- `readDistance()` triggers the sensor, measures the echo time with `pulseIn()` using a 25 ms timeout, and returns the distance in cm, capped at 200.
- **Status logic** in `loop()` reads the sensor every 100 ms and sets the status to 2 if the distance is under 30 cm, 1 if it is under 60 cm, and 0 otherwise. The buzzer pin is driven high only when the status is 2.
- `sendThingSpeak()` sends an HTTP GET request to the ThingSpeak update API with the Write API Key, distance (field 1) and status (field 2). The server replies with the entry number, which is printed on the Serial Monitor to confirm the upload.
- **Web server:** `WebServer` serves the dashboard page at `/` and the latest distance at `/d`. The page uses JavaScript and the HTML canvas to redraw the radar, the robot icon, the obstacle icon (a brick for normal obstacles, a barrier when in danger), the colour zones and a live line graph.

The complete code is in the appendix.

8. Results

The system worked as intended. When the hand or an object was moved towards the sensor, the distance on the dashboard decreased, the banner changed from green to yellow to red, the obstacle icon moved closer to the robot on the radar, and the buzzer sounded under 30 cm. When the object was removed, the dashboard returned to "Path clear".

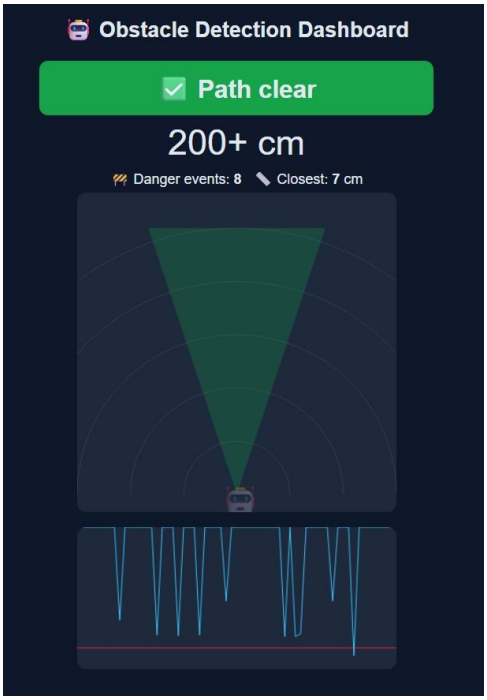


Figure 1: Live dashboard with no obstacle in range.

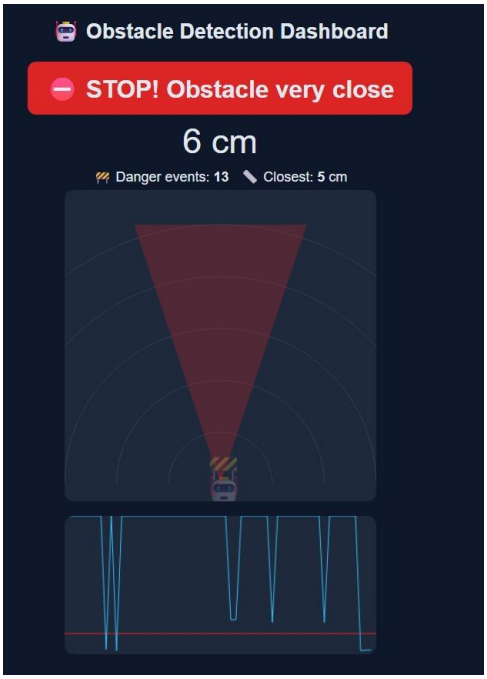


Figure 2: Live dashboard when an obstacle is very close. The red zone, obstacle icon and danger counter react in real time.



Figure 3: Serial Monitor confirming successful uploads to ThingSpeak.



Figure 4: ThingSpeak history graphs. The dips in the distance graph match the times an object was brought close to the sensor, and the status graph rises at the same moments.

The channel collected 55 entries during testing, and the dashboard recorded the closest distance of 5 cm.

9. Challenges Faced

- **COM port busy or not detected:** fixed by changing the USB cable, selecting the correct port again and closing other programs using it.
- **Hotspot would not connect:** the iPhone name contained an apostrophe that did not match the code. Renaming the phone, fixed it. The ESP32 also supports only 2.4 GHz, so "Maximize Compatibility" had to be turned on.
- **Echo pin voltage:** the sensor outputs 5 V while the ESP32 accepts 3.3 V, solved with the resistor divider.
- **ThingSpeak update limit:** only one update per 15 seconds is allowed, so the cloud graph is slower than the live dashboard. This is why the dashboard is served directly from the ESP32.
- **Empty chart:** the ThingSpeak chart appeared blank at first and showed data after refreshing and adjusting the chart settings.

10. Limitations

- The system depends on a stable WiFi connection. The Serial Monitor occasionally showed failed uploads (HTTP -1 and -11) when the hotspot connection dropped.
- Soft or angled surfaces reflect sound poorly, which causes sudden spikes to 200 cm in the readings.
- The live dashboard works only for devices on the same WiFi network as the ESP32.
- The prototype was tested with hand and objects and is not yet mounted on a motorised vehicle.

11. Future Scope

- Mount the system on a motorised car and stop the motors automatically when an obstacle is under 30 cm (using a motor driver such as the L298N).
- Sweep the sensor with a servo motor for a 180-degree radar view.
- Add the DHT11 sensor to correct the distance for temperature.
- Send phone alerts and publish a public dashboard.
- Use machine learning on the stored data to classify obstacles or predict collisions.

12. Conclusion

We built a working IoT prototype that measures distance with an ultrasonic sensor, makes the safe, warning or danger decision on the ESP32 at the edge, sounds a buzzer immediately, uploads the data to ThingSpeak for storage and history graphs, and shows a live dashboard with a robot, obstacle icons and colour-coded alerts. This meets the objective of visualizing obstacle detection in real time and gives us hands-on experience with microcontrollers, sensor interfacing, WiFi communication, and cloud platforms.

Appendix: Arduino Code

```
#include <WiFi.h>
#include <WebServer.h>
#include <HTTPClient.h>

const char* WIFI_SSID = "YOUR_HOTSPOT_NAME";
const char* WIFI_PASS = "YOUR_HOTSPOT_PASSWORD";
String TS_KEY = "YOUR_WRITE_API_KEY";

#define TRIG 5
#define ECHO 18
#define BUZZ 23

WebServer server(80);
int dist = 200;
int status = 0;
unsigned long lastRead = 0, lastTs = 0;

const char PAGE[] PROGMEM = R"rawliteral(
<!DOCTYPE html><html><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Robot Obstacle Radar</title>
<style>
body{margin:0;background:#0f172a;color:#e2e8f0;font-family:sans-serif;text-
align:center}
#status{font-size:28px;font-weight:bold;padding:12px;margin:8px auto;border-
radius:10px;max-width:420px}
.safe{background:#16a34a}.warn{background:#ca8a04}.danger{background:#dc2626}.
off{background:#475569}
canvas{background:#1e293b;border-radius:10px;margin:6px;max-width:95%}
.big{font-size:40px;margin:6px}
</style></head><body>
<h2>🚨 Obstacle Detection Dashboard</h2>
<div id="status" class="off">Connecting...</div>
<div class="big"><span id="dist">--</span> cm</div>
<div>🚧 Danger events: <b id="cnt">0</b>   📏 Closest: <b id="min">--
</b> cm</div>
<canvas id="view" width="360" height="360"></canvas><br>
<canvas id="chart" width="360" height="160"></canvas>
<script>
const MAX=200,hist=[];let prev=0,count=0,minD=999;
const v=document.getElementById('view').getContext('2d');
const c=document.getElementById('chart').getContext('2d');
const col=d=>d<30?'#dc2626':d<60?'#ca8a04':'#16a34a';
function draw(d){
  v.clearRect(0,0,360,360);v.strokeStyle='#334155';
  for(let
r=60;r<=300;r+=60){v.beginPath();v.arc(180,340,r,Math.PI,0);v.stroke();}

v.fillStyle=col(d)+'44';v.beginPath();v.moveTo(180,340);v.lineTo(80,40);v.line
To(280,40);v.fill();
  v.font='32px sans-serif';v.fillText('🚍',162,358);
  if(d<MAX){const y=340-(d/MAX)*300;v.font='34px sans-serif';
    v.fillText(d<30?'🚧':'🧱',160,y);
    v.strokeStyle=col(d);v.setLineDash([5,5]);v.beginPath();
    v.moveTo(180,330);v.lineTo(180,y+10);v.stroke();v.setLineDash([]);}
}
function chart(){
  c.clearRect(0,0,360,160);
  c.strokeStyle='#dc2626';c.beginPath();c.moveTo(0,160-
30/MAX*160);c.lineTo(360,160-30/MAX*160);c.stroke();
  c.strokeStyle='#38bdf8';c.beginPath();
  hist.forEach((d,i)=>{const x=i*6,y=160-
Math.min(d,MAX)/MAX*160;i?c.lineTo(x,y):c.moveTo(x,y)});
  c.stroke();
}
async function tick(){
  const s=document.getElementById('status');
  try{
    const d=parseInt(await (await fetch('/d')).text());
    document.getElementById('dist').innerText=d>=MAX?'200+':d;
    if(d<30){s.className='danger';s.innerText='⛔ STOP! Obstacle very close'}
    else if(d<60){s.className='warn';s.innerText='⚠️ Approaching obstacle'}
    else{s.className='safe';s.innerText='✅ Path clear'}
    const st=d<30?2:0;
    if(st==2&&prev!=2){count++;document.getElementById('cnt').innerText=count}
    prev=st;if(d<minD){minD=d;document.getElementById('min').innerText=d}
    hist.push(d);if(hist.length>60)hist.shift();
    draw(d);chart();
  }catch(e){s.className='off';s.innerText='Offline...'}
  setTimeout(tick,300);
}
tick();
</script></body></html>
)rawliteral";

int readDistance() {
  digitalWrite(TRIG, LOW); delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long t = pulseIn(ECHO, HIGH, 25000);
  if (t == 0) return 200;
  return min(200, (int)(t * 0.034 / 2));
}

void sendThingSpeak() {
  HTTPClient http;
  String url = "http://api.thingspeak.com/update?api_key=" + TS_KEY +
               "&field1=" + String(dist) + "&field2=" + String(status);
  http.begin(url);
  int code = http.GET();
  Serial.println("ThingSpeak HTTP " + String(code) + " | entry: " +
http.getString());
  http.end();
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT); pinMode(BUZZ, OUTPUT);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nConnected! OPEN THIS IN BROWSER: http://" +
WiFi.localIP().toString());
  server.on("/", []() { server.send_P(200, "text/html", PAGE); });
  server.on("/d", []() { server.send(200, "text/plain", String(dist)); });
  server.begin();
}

void loop() {
  server.handleClient();
  unsigned long now = millis();
  if (now - lastRead > 100) {
    lastRead = now;
    dist = readDistance();
    status = dist < 30 ? 2 : dist < 60 ? 1 : 0;
    digitalWrite(BUZZ, status == 2);
  }
  if (WiFi.status() == WL_CONNECTED && now - lastTs > 16000) {
    lastTs = now;
    sendThingSpeak();
  }
}
```

ThingSpeak channel ID: 3564235